

รายงานโครงงาน

การพัฒนาระบบสั่งอาหารชุมชน huaychan

พื้นที่กลางสำหรับคนหาร้าน สั่งอาหาร และติดต่อร้านค้าในชุมชนห้วยชัน นาฝาย
และบริเวณมหาวิทยาลัยราชภัฏชัยภูมิ

จัดทำโดย

รัชชานนท์ ผมไผ

รหัสนักศึกษา 681102118

สาขาวิทยาการคอมพิวเตอร์

เสนอต่อ

อาจารย์ ฤทธิชัย ผานาค

รายวิชา [ชื่อและรหัสวิชา]

มหาวิทยาลัยราชภัฏชัยภูมิ

ภาคเรียน [ระบุ] ปีการศึกษา [ระบุ]

เว็บไซต์โครงงาน <https://huaychan.web.app>

ฉบับอ้างอิงโค้ดและข้อมูลที่ตรวจเมื่อวันที่ 17 กันยายน 2569

บทคัดย่อ

โครงการ huaychan

มีวัตถุประสงค์เพื่อพัฒนาพื้นที่กลางสำหรับร้านอาหารและเครื่องดื่มในชุมชน

ให้ผู้ซื้อค้นหาร้านและเมนู สร้างรายการสั่งซื้อ ติดตามสถานะ และติดต่อร้านค้าได้ในระบบเดียว
แนวคิดเริ่มจากการสังเกตของผู้จัดทำว่าพ่อค้าแม่ค้าใกล้บ้านใช้ Facebook

ประกาศขายสินค้าอยู่เป็นประจำ

จึงต้องการจัดข้อมูลร้านและเมนูให้อยู่ในรูปแบบที่ค้นหาได้สะดวกและเชื่อมกับขั้นตอนการสั่งซื้อ

ระบบแบ่งผู้ใช้ออกเป็นลูกค้า ร้านค้า และผู้ดูแลระบบ ร้านค้าเป็นผู้ขาย ปิ้งอาหาร รับเงิน
และจัดส่งเอง ส่วน huaychan ดูแลระบบสารสนเทศและช่วยประสานงาน ใช้ Next.js และ
React สร้างส่วนติดต่อผู้ใช้ ใช้ TypeScript อธิบายโครงสร้างข้อมูล และเชื่อม Firebase
สำหรับบัญชีผู้ใช้ ฐานข้อมูล และฟังก์ชันฝั่งเซิร์ฟเวอร์ มีส่วนรองรับ LINE LIFF และแอป
Android ผ่าน Capacitor

การศึกษาในรายงานใช้การตรวจสอบซอร์สโค้ด การสังเกตหน้าสาธารณะของเว็บไซต์

และการรันชุดทดสอบย่อย 20 กรณี ซึ่งผ่านทั้งหมดในการตรวจครั้งนี้

ผลดังกล่าวครอบคลุมตรรกะที่ทดสอบเฉพาะส่วน ไม่ใช่ผลยืนยันการทำงานจริงครบทุกขั้นตอน
บันทึกตรวจสอบความพร้อมของโครงการยังระบุประเด็นที่ต้องแก้ เช่น การคำนวณราคาฝั่งลูกค้า
การสั่งซื้อ และการควบคุมการจบออเดอร์

ผลที่ได้คือระบบต้นแบบที่แสดงแนวทางนำการขายของในชุมชนมาจัดเป็นหน้าร้านออนไลน์

มีโครงสร้างรองรับการสั่งซื้อและการสื่อสารโดยตรง

รายงานยังเสนอแนวทางตรวจรับและประเมินกับผู้ใช้จริงก่อนเปิดใช้งานทั่วไป

โดยยังไม่มีข้อมูลเพียงพอที่จะสรุปว่าระบบเพิ่มยอดขายหรือลดเวลาสั่งซื้อได้เท่าใด

คำสำคัญ ร้านค้าชุมชน ระบบสั่งอาหาร เว็บแอปพลิเคชัน Firebase huaychan

หมายเหตุฉบับอัปเดต ระหว่างจัดทำรายงานโค้ดได้รับการปรับปรุง backend checkout และการกันค่าขอซ้ำ ตารางข้อค้นพบเดิมใช้เป็นประวัติการพัฒนา ส่วนสถานะ deploy อ้างตามบันทึกของโครงการและยังต้องตรวจรับบนอุปกรณ์จริง

ขอบเขตหลักฐานและวิธีอ่านรายงาน

รายงานฉบับนี้อธิบายระบบจากโค้ดในโฟลเดอร์ D:/tonpalmpjproject และหน้าสาธารณะของเว็บไซต์ในวันที่ตรวจ ไม่ใช่บันทึกเวลาทำงานย้อนหลังทุกวัน ลำดับการพัฒนาในบทที่ 3 เป็นการเรียงเรียงขั้นตอนทางวิศวกรรมจากองค์ประกอบที่มีอยู่ เพื่อให้ผู้อ่านเข้าใจและทำซ้ำได้ โดยไม่ได้อ้างว่าเป็นลำดับวันที่ผู้จัดทำลงมือจริง

ที่มาของปัญหาและเหตุผลเลือกโครงการมาจากคำอธิบายของผู้จัดทำ ไม่ใช่ผลสำรวจตัวอย่างประชากร การกล่าวถึงโพสต์ใน Facebook จึงเป็นข้อสังเกตในบริบทใกล้เคียง ไม่ใช่ข้อสรุปทางสถิติของร้านค้าทั้งหมด

คำว่า “พบในโค้ด” หมายถึงมีส่วนประกอบหรือเงื่อนไขนั้นในไฟล์ที่ตรวจ ส่วนคำว่า “ทดสอบผ่าน” หมายถึงผ่านเฉพาะวิธีทดสอบที่ระบุ ผลจาก LAUNCH_READINESS_REVIEW.md แยกเป็นหลักฐานเดิมในโครงการ และไม่ได้ถือว่าได้รับทดสอบซ้ำทุกกรณีเพื่อจัดทำรายงานนี้

ผู้จัดทำคือ รัชชานนท์ ผมไผ รหัสนักศึกษา 681102118 สาขาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยราชภัฏชัยภูมิ เสนอต่ออาจารย์ ฤทธิชัย ผานาค การเลือกโครงการตามข้อกำหนดรายวิชาสมควรอธิบายว่าเสนอประเด็นจากชุมชนของตนเอง หากต้องมียานจากมหาวิทยาลัยไทยเป็นงานที่เกี่ยวข้อง จะต้องเพิ่มเอกสารต้นฉบับที่ตรวจสอบได้ก่อนส่งฉบับสมบูรณ์

ข้อมูลราคาในตัวอย่างคำนวณเป็นข้อมูลสมมติ ไม่ใช่ยอดขายจริง ภาพหน้าจจะเป็นภาพหน้าเว็บสาธารณะและอาจเปลี่ยนไปตามเวลาที่เปิด ระบบจริงไม่ได้ถูกสร้างออเดอร์ ส่งข้อความ เต็มเครดิต หรือเปลี่ยนการตั้งค่าระหว่างการจัดทำรายงาน

บทคัดย่อ	2
ขอบเขตหลักฐานและวิธีอ่านรายงาน	3
สารบัญ	4
บทที่ 1 บทนำและการกำหนดปัญหา	6
1.3 ปัญหา วัตถุประสงค์ และเกณฑ์ตรวจรับ	7
1.4 ขอบเขตและผู้เกี่ยวข้อง	8
บทที่ 2 แนวคิดและเทคโนโลยีที่เกี่ยวข้อง.....	9
2.5 เทคโนโลยีที่ใช้ในโครงการ	10
ฐานข้อมูลที่ปรับหน้าจอตตามการเปลี่ยนแปลง	10
งานหลายรายการที่ต้องสำเร็จร่วมกัน.....	10
2.6 บัญชีผู้ใช้ แอปมือถือ และงานที่เกี่ยวข้อง	11
บทที่ 3 การวิเคราะห์ ออกแบบ และพัฒนาระบบ	12
3.4 ความต้องการเชิงหน้าที่และเงื่อนไขการใช้งาน	13
3.5 สถาปัตยกรรมและโครงสร้างไฟล์.....	14
3.6 แบบจำลองข้อมูลและความสัมพันธ์.....	15
3.7 พจนานุกรมข้อมูลออเดอร์	16
3.8 เส้นทางการสั่งซื้อและสถานะงาน.....	17
3.9 ตะกร้า ตัวเลือก และตัวอย่างการคำนวณ	18
3.10 บัญชีผู้ใช้และการกำหนดสิทธิ์	19
3.11 แชน ด่าแหน่ง และการแจ้งเตือน	20
3.12 การจับออเดอร์และการสรุปยอดขาย	21
3.13 การติดตั้งและเตรียมสภาพแวดล้อม	22
3.14 การเผยแพร่และดูแลหลังติดตั้ง	23
บทที่ 4 ผลการพัฒนาและการทดสอบ	24
4.2 ผลการตรวจสอบองค์ประกอบระบบ	25
4.3 ผลชุดทดสอบที่รันในการจัดทำรายงาน.....	26
4.4 ประเด็นจากบันทึกตรวจสอบความพร้อม	27
4.5 แผนทดสอบระบบครบเส้นทาง	28
บทที่ 5 สรุป อภิปรายผล และแนวทางพัฒนา.....	29
5.6 การประเมินกับชุมชนและการต่อยอด	30
บรรณานุกรม	31

ภาคผนวก ก คู่มือใช้งานสำหรับลูกค้า	32
ภาคผนวก ก คู่มือสำหรับร้านค้าและผู้ดูแล	33
ภาคผนวก ข โค้ดตะกร้าและการจัดกลุ่มร้าน	34
ภาคผนวก ข โค้ดจองเลขและสร้างออเดอร์	35
ภาคผนวก ข โค้ดสิทธิ์ของออเดอร์	36
ภาคผนวก ข โค้ด transaction เมื่อจบออเดอร์.....	37
ภาคผนวก ข โค้ดสรุปยอดตามธุรกรรมจริง	38
ภาคผนวก ค แนวคำถามสำหรับนำเสนอโครงการ	39
ภาคผนวก ค แบบบันทึกทดสอบและประเมิน	40

บทที่ 1 บทนำและการกำหนดปัญหา

1.1 ความเป็นมา เดิมรายวิชากำหนดแนวทางให้เลือกศึกษาโครงการจากมหาวิทยาลัยต่าง ๆ ในประเทศไทย ผู้จัดทำยังไม่พบหัวข้อที่ตรงกับความสนใจ

จึงกลับมาพิจารณาสิ่งที่พบในชีวิตประจำวัน ได้แก่

การโพสต์ขายอาหารและเครื่องดื่มของร้านค้าภายในชุมชนบน Facebook

แนวคิดจึงเริ่มจากปัญหาใกล้ตัวและความต้องการทำระบบที่สัมพันธ์กับผู้ใช้ที่พบเห็นได้จริง

โพสต์ขายของช่วยให้คนในชุมชนรับรู้ว่ามีร้านใดขายอะไร แต่ข้อมูลอาจอยู่ต่างโพสต์ ต่างวัน

หรือต่างช่องทาง ผู้ซื้ออาจต้องค้นหาราคา สอบถามเมนู

และแจ้งสถานที่จัดส่งในบทสนทนาหลายรอบ จากข้อสังเกตนี้

ผู้จัดทำเห็นโอกาสรวบรวมข้อมูลร้าน เมนู และรายการสั่งซื้อให้อยู่ในพื้นที่เดียว

โดยยังเปิดให้ผู้ซื้อคุยกับร้านโดยตรง

1.2 แนวคิดโครงการ huaychan

เป็นพื้นที่กลางของชุมชนสำหรับพบร้านค้าและส่งข้อมูลการสั่งซื้อ ร้านแต่ละแห่งมีข้อมูลหน้าร้าน

เมนู และสถานะเปิดรับออเดอร์ ลูกค้าเลือกสินค้าและติดตามงานของร้านได้

โครงการเน้นพื้นที่ห้วยชัน นาฝาย และบริเวณมหาวิทยาลัยราชภัฏชัยภูมิตามที่เว็บไซต์สื่อสาร

ความหมายของการเป็นตัวกลางในรายงานคือการจัดการข้อมูลและช่องทางติดต่อ

ไม่ใช่การทำอาหารหรือจัดทีมขนส่งของแพลตฟอร์ม ผู้ดูแลยังมีหน้าที่ดูแลระบบ บัญชี

สิทธิ์เข้าถึง และการประสานปัญหาที่เกี่ยวข้องกับระบบ จึงควรอธิบายบทบาทนี้ให้ชัด

แทนการกล่าวว่าแพลตฟอร์มไม่มีความรับผิดชอบใดเลย

จุดมุ่งหมายของโครงการสรุปได้ว่า ร้านเล็ก ๆ ควรมีหน้าร้านออนไลน์ที่คนแถวบ้านค้นพบได้ง่าย

และการสั่งซื้อควรเชื่อมคนซื้อกับคนขายได้สะดวก ข้อความนี้เป็นเจตนารมณ์การออกแบบ

ส่วนผลต่อรายได้หรือชุมชนต้องเก็บหลักฐานเพิ่มเติมภายหลัง

1.3 ปัญหา วัตถุประสงค์ และเกณฑ์ตรวจรับ

ปัญหาหลักที่โครงการเลือกแก้คือการกระจายตัวของข้อมูลและขั้นตอนการสั่งซื้อ ตัวอย่างเช่น ลูกค้าเห็นรูปอาหารแต่ไม่ทราบว่าร้านยังเปิดอยู่หรือไม่ เมื่อตัดสินใจซื้อยังต้องส่งรายละเอียดเมนูจำนวน และที่อยู่ให้ครบ ระบบจึงควรจัดข้อมูลเหล่านี้ให้เป็นรายการที่ตรวจทานได้ก่อนส่งถึงร้าน

ปัญหาที่เริ่มต้น	วัตถุประสงค์	เกณฑ์ตรวจรับที่เสนอ
ข้อมูลร้านและเมนูกระจาย	รวมร้านและเมนูให้ค้นหาได้	ค้นหาชื่อร้านหรือเมนูทดสอบและเปิดรายละเอียดได้
รายละเอียดออเดอร์ไม่เป็นรูปแบบเดียวกัน	สร้างตะกร้าและสรุปยอดแยกร้าน	จำนวน ตัวเลือก และยอดรวมตรงกับข้อมูลทดสอบ
ต้องถามความคืบหน้าซ้ำ	แสดงสถานะและแชทตามออเดอร์	บัญชีคู่สั่งซื้อเห็นสถานะและข้อความของรายการเดียวกัน
ร้านต้องจัดการหลายรายการ	มีหน้ารับงานและประวัติ	ร้านเห็นเฉพาะงานของตนและแยกงานจบแล้วได้
การจัดการสิทธิ์มีความเสี่ยง	แยกบทบาทและควบคุมข้อมูล	บัญชีไม่มีสิทธิ์ต้องอ่านหรือแก้ไขข้อมูลส่วนตัวไม่ได้

เกณฑ์ด้านบนเป็นเกณฑ์ที่ควรใช้ตรวจรับเพิ่มเติม ไม่ใช่การรับรองว่าผ่านทั้งหมดแล้ว โดยเฉพาะยอดเงินจริง การทำงานพร้อมกันหลายค่าขอ และขอบเขตสิทธิ์ต้องตรวจที่ฐานข้อมูลและฟังก์ชันฝั่งเซิร์ฟเวอร์ ไม่พิจารณาเพียงว่าหน้าจอแสดงถูกต้อง

1.4 ขอบเขตและผู้เกี่ยวข้อง

ขอบเขตด้านลูกค้าครอบคลุมการดูร้านและเมนู เข้าสู่ระบบ เลือกตัวเลือกอาหาร ระบุจำนวนและหมายเหตุ กรอกข้อมูลติดต่อและที่อยู่ เลือกวิธีชำระเงิน ส่งออเดอร์ และติดต่อร้านผ่านแชท ข้อมูลตำแหน่งเป็นตัวช่วยระบุจุดส่ง แต่รายละเอียดข้อความยังสำคัญ เช่น อาคาร ชั้น หรือจุดนัดรับ

ขอบเขตร้านค้าครอบคลุมข้อมูลหน้าร้าน เมนู สถานะพร้อมขาย การรับงาน เปลี่ยนสถานะ สื่อสารกับลูกค้า และตรวจประวัติหรือสรุปยอด ส่วนการปรุงอาหาร การรับเงิน และการจัดส่งเป็นงานของร้าน ระบบไม่ได้แสดงหลักฐานว่ามีเครือข่ายคนขับอิสระของแพลตฟอร์ม

ขอบเขตผู้ดูแลระบบครอบคลุมข้อมูลร้านและผู้ใช้ สิทธิ์ การติดตามออเดอร์ การตั้งค่า ค่าบริการหรือเครดิต และการช่วยประสานเหตุขัดข้อง ความสามารถบางส่วนมีอยู่ในโค้ดแต่ยังต้องตรวจรับการเรียกใช้กับบริการจริงตามข้อกำหนดในบทที่ 4

ผู้เกี่ยวข้อง	หน้าที่ในกระบวนการ	ข้อมูลที่จำเป็น
ลูกค้า	เลือกรายการ ตรวจยอด แจ้งจุดส่ง และชำระกับร้าน	ชื่อ เบอร์โทร ที่อยู่ รายการซื้อ
ร้านค้า	ดูแลเมนู รับงาน ปรุง รับเงิน และส่งอาหาร	ออเดอร์ของร้าน จุดส่ง และบทสนทนา
ผู้ดูแล	ดูแลระบบ สิทธิ์ และช่วยประสานงาน	ข้อมูลตามสิทธิ์สำหรับแก้ปัญหาและตรวจสอบ

ประโยชน์ที่คาดหวังคือช่วยให้ร้านมีหน้าร้านออนไลน์และให้ลูกค้าค้นหาสินค้าใกล้เคียงได้สะดวก ไม่ใช้จำนวนร้าน ยอดขาย หรือคะแนนบนหน้าเว็บเป็นหลักฐานประเมินความสำเร็จ เพราะยังไม่แยกได้ว่ารายการใดเป็นข้อมูลทดลองหรือการใช้งานจริง

บทที่ 2 แนวคิดและเทคโนโลยีที่เกี่ยวข้อง

2.1 ระบบสารสนเทศสำหรับการสั่งซื้อ การสั่งซื้อหนึ่งครั้งเกี่ยวข้องกับข้อมูลหลายกลุ่ม เช่น ผู้ซื้อ ร้านค้า รายการสินค้า ที่อยู่ ยอดเงิน และสถานะ

ระบบสารสนเทศนำข้อมูลเหล่านี้มาจัดรูปแบบให้จัดเก็บและค้นคืนได้

รายการออเดอร์จึงเป็นศูนย์กลางเชื่อมขั้นตอนตั้งแต่ยืนยันสินค้าไปจนถึงการติดตามผล

2.2 การพัฒนาเว็บด้วยองค์ประกอบย่อย ในโครงการ React

ทำหน้าที่จัดส่วนติดต่อเป็นองค์ประกอบ เช่น หน้าร้าน ตะกร้า และกล่องแชท

การแบ่งองค์ประกอบช่วยให้แต่ละส่วนมีหน้าที่ชัดเจน ส่วน Context ใช้ส่งสถานะร่วม ได้แก่ บัญชีผู้ใช้และตะกร้าไปยังส่วนที่ต้องใช้ โดยไม่ต้องส่งค่าผ่านทุกชั้นของหน้าจอ

2.3 แบบจำลองข้อมูลด้วย TypeScript ไฟล์ `src/types/index.ts` กำหนดชนิดข้อมูลของ `UserProfile`, `Shop`, `MenuItem`, `Order` และ `ChatMessage`

ช่วยให้ผู้พัฒนาเห็นว่าข้อมูลแต่ละกลุ่มควรมีช่องใดและใช้ค่าประเภทใด อย่างไรก็ตาม ชนิดข้อมูลในโปรแกรมไม่ใช่กลไกป้องกันค่าขอลอม จึงต้องมีการตรวจที่ backend หรือ Firestore Rules อีกชั้นหนึ่ง

2.4 Next.js และการเผยแพร่แบบ static export โครงการระบุ Next.js 16.3.5 และตั้ง output เป็น export เมื่อ build จะได้ไฟล์สำหรับเผยแพร่ใน out ตามคู่มือที่ติดตั้งมากับรุ่นนี้ เว็บไซต์จึงใช้ Firebase Hosting เซิร์ฟไฟล์ ส่วนงานข้อมูลและงานที่ต้องเชื่อถือได้เชื่อม Firebase แยกต่างหาก ไม่ได้อาศัย Next.js server ที่ทำงานตลอดเวลา

2.5 เทคโนโลยีที่ใช้ในโครงการ

เทคโนโลยี	หลักฐานเวอร์ชันใน package.json	หน้าที่ในโครงการ
Next.js / React	16.3.5 / 19.2.8	โครงสร้างหน้าและส่วนติดต่อ
TypeScript	^5	อธิบายชนิดข้อมูลและตรวจโค้ด
Tailwind CSS	^4	รูปแบบหน้าจอ
Firebase SDK	^12.19.0	บัญชี ฐานข้อมูล และเรียก Functions
LINE LIFF	^2.31.0	เชื่อมการเข้าสู่ระบบ LINE
Capacitor	Core ^8.5.2	เชื่อมเว็บกับความสามารถ Android
Leaflet / React Leaflet	^1.9.4 / ^5.0.0	แผนที่และเลือกตำแหน่ง

เครื่องหมาย ^ ระบุช่วงเวอร์ชันที่ยอมรับใน manifest

ไม่ได้หมายความว่าเครื่องทุกเครื่องติดตั้งเวอร์ชันเดียวกัน การทำซ้ำควรใช้ package-lock.json และ npm ci เพื่อให้ตรงกับชุด dependency ที่ล็อกไว้

ฐานข้อมูลที่ปรับหน้าจอตามการเปลี่ยนแปลง

เอกสาร Firebase อธิบายว่า onSnapshot ส่งข้อมูลปัจจุบันให้ callback

และแจ้งอีกเมื่อข้อมูลเปลี่ยน ในโครงการนารูปแบบนี้มาใช้กับข้อมูลที่ต้องเปลี่ยนตามเหตุการณ์ เช่น แชนและสถานะผู้ใช้ [1] การแสดงผลจึงไม่จำเป็นต้องให้ผู้ใช้กดโหลดใหม่ทุกครั้ง แต่ต้องจัดการยกเลิก listener เมื่อเลิกใช้งาน

งานหลายรายการที่ต้องสำเร็จร่วมกัน

Transaction ใช้รวมการอ่านและเขียนที่ต้องสัมพันธ์กัน เช่น

ปิดออเดอร์พร้อมปรับเครดิตและบันทึกธุรกรรม หากมีการเปลี่ยนข้อมูลที่เกี่ยวข้องพร้อมกัน ระบบอาจเรียกตรรกะ transaction เข้า จึงควรออกแบบให้ผลถูกต้องเมื่อมีการลองใหม่ [2]

2.6 บัญชีผู้ใช้ แอปมือถือ และงานที่เกี่ยวข้อง

การเข้าสู่ระบบด้วย LINE ในโค้ดรับ ID token แล้วส่งไปให้ signInWithLine ตรวจสอบฟังก์ชันเชิร์ฟเวอร์ก่อนนำ custom token มาเข้าสู่ Firebase การแยกขั้นตอนนี้มีความสำคัญเพราะข้อมูลชื่อหรือรหัสผู้ใช้ที่ส่งมาจากหน้าจอเพียงอย่างเดียวไม่ควรเป็นหลักฐานยืนยันตัวตนเอกสาร LINE แนะนำให้ส่ง token ไปตรวจสอบเมื่อใช้ข้อมูลผู้ใช้บนเชิร์ฟเวอร์ [3]

Capacitor เป็นชั้นเชื่อมแอปเว็บกับแพลตฟอร์มมือถือ [4] โครงการนี้มีไฟล์เดอริแอนด์ android และปลั๊กอินสำหรับตำแหน่ง การแจ้งเตือน และสถานะแอป ใน capacitor.config.ts ตั้ง server.url ไปยังเว็บไซต์จริง จึงต้องแยกระหว่างความสำเร็จของการ build เว็บกับการทดสอบบนเครื่อง Android จริง รวมถึงกรณีแอปอยู่เบื้องหลังและการกลับจาก LINE

งานที่เกี่ยวข้องในรายงานนี้ใช้เอกสารเทคโนโลยีจากผู้พัฒนาและหลักฐานระบบของโครงการยังไม่ได้รับรายชื่อโครงการจากมหาวิทยาลัยไทยที่ผู้จัดทำเคยพิจารณา จึงไม่ระบุชื่อผู้วิจัยชื่อมหาวิทยาลัย หรือผลวิจัยโดยสมมติ หากรายวิชากำหนดให้มีส่วนนี้ควรนำเอกสารที่อาจารย์ยอมรับมาเปรียบเทียบโดยตรง

ประเด็นเปรียบเทียบ	การขายผ่านโพสต์ตามข้อสังเกต	แนวทางของ huaychan
การค้นข้อมูล	ค้นโพสต์และสอบถามรายละเอียด	รวมข้อมูลร้านและเมนู
รายละเอียดการสั่ง	ส่งข้อความอิสระ	มีตะกร้าและข้อมูลออเดอร์
การติดต่อ	บทสนทนาของแต่ละช่องทาง	แชทผูกกับรหัสออเดอร์
การจัดส่ง	ตกลงกับร้าน	ร้านจัดส่งเองตามข้อมูลออเดอร์

ตารางนี้เป็นการเปรียบเทียบแนวทางออกแบบจากโจทย์ของผู้จัดทำไม่ใช่ผลการทดลองเปรียบเทียบกับ Facebook และไม่ใช่ข้ออ้างว่าทุกกลุ่มขายของมีวิธีทำงานเหมือนกัน

บทที่ 3 การวิเคราะห์ ออกแบบ และพัฒนาระบบ

3.1 ขั้นตอนเริ่มต้น เริ่มจากระบุเหตุการณ์ที่ต้องการช่วยเหลือ ได้แก่ ลูกค้ามองหาร้านในชุมชนและร้านต้องรับรายละเอียดการสั่งซื้อให้ครบ จากนั้นเขียนขอบเขตว่าระบบรับผิดชอบการจัดการข้อมูลและการสื่อสาร ส่วนการทำอาหารและส่งสินค้าเป็นงานของร้าน การตกลงขอบเขตตั้งแต่ต้นช่วยไม่ให้เกิดพัฒนาความสามารถด้านคนขับหรือการรับเงินกลางโดยไม่มีความต้องการรองรับ

3.2 การแปลงปัญหาเป็นความต้องการ แยกบทบาทเป็นลูกค้า ร้านค้า และแอดมิน เขียนเหตุการณ์ของแต่ละบทบาท เช่น ลูกค้าสั่งอาหารหนึ่งร้าน ร้านรับออเดอร์ แอดมินช่วยตรวจสอบสถานะ แล้วกำหนดข้อมูลเข้า ผลลัพธ์ และเงื่อนไขพิเศษของแต่ละเหตุการณ์ ตัวอย่างคือการยืนยันออเดอร์ต้องมีผู้ใช้ เบอร์โทร และรายละเอียดที่อยู่

3.3 การพัฒนาเป็นรอบ ลำดับที่เหมาะสมสำหรับทำซ้ำคือสร้างแบบจำลองข้อมูลและหน้าร้านก่อน ต่อด้วยบัญชีและตะกร้า จากนั้นเพิ่มออเดอร์ สถานะ แชนแนล และงานดูแลระบบ ทดสอบที่ละเส้นทางแล้วจึงเชื่อมการแจ้งเตือนและ Android ลำดับนี้อธิบายความสัมพันธ์ของงาน ไม่ใช่การยืนยันวันเวลาที่พัฒนาในอดีต

ขั้นตอน	สิ่งที่ควรได้	หลักฐานในโครงการ
วิเคราะห์	ขอบเขตและบทบาท	หน้า about และชนิด UserRole
ออกแบบ	ข้อมูลและการไหลของงาน	src/types/index.ts
พัฒนา	หน้าจอและบริการ	src/components และ functions
ตรวจสอบ	กรณีทดสอบและข้อจำกัด	tests และบันทึก readiness
เผยแพร่	ไฟล์เว็บและแอปทดลอง	firebase.json และ android

3.4 ความต้องการเชิงหน้าที่และเงื่อนไขการใช้งาน

รหัส	ความต้องการ	เงื่อนไขสำคัญ
FR01	ค้นหาและเปิดร้าน	ข้อมูลชื่อ ราคา และความพร้อมต้องอ่านง่าย
FR02	จัดการบัญชี	ยืนยันตัวตนก่อนทำรายการที่ต้องระบุเจ้าของ
FR03	ตระกร้าหลายร้าน	แยกรายการและค่าส่งตามร้าน
FR04	สร้างออเดอร์	ตรวจสอบผู้ซื้อ ร้าน รายการ และยอดก่อนบันทึก
FR05	ติดตามสถานะ	ใช้สถานะที่กำหนดและจำกัดผู้เปลี่ยน
FR06	แชทตามออเดอร์	เข้าถึงได้เฉพาะผู้เกี่ยวข้อง
FR07	จัดการเมนู	ร้านแก้ไขข้อมูลร้านตนเองตามสิทธิ์
FR08	รายงานยอดขาย	แยกยอดอาหาร ค่าส่ง และค่าบริการจริง
FR09	ผู้ดูแลระบบ	จัดการสิทธิ์และตรวจปัญหาย้อนหลัง

ความต้องการที่ไม่ใช่หน้าที่โดยตรง ได้แก่ อ่านได้บนมือถือ แฉงความผิดพลาดอย่างเข้าใจง่าย ป้องกันการเข้าถึงข้อมูลข้ามร้าน และรักษาความถูกต้องเมื่อผู้ใช้กดซ้ำ การวัดความเร็วควรใช้เครื่อง เครือข่าย และจำนวนข้อมูลที่ระบุขีด ก่อนรายงานเป็นตัวเลข

ระบบควรแสดงความแตกต่างระหว่าง “กำลังส่งคำขอ” “บันทึกสำเร็จ” และ “แจ้งเตือนร้านสำเร็จ” เพราะออเดอร์อาจบันทึกได้แม้บริการแจ้งเตือนผิดพลาด การผูกความสำเร็จทั้งหมดเข้ากับข้อความเดียวจะทำให้ผู้ใช้ตีความผิดและอาจสั่งซ้ำ

3.5 สถาปัตยกรรมและโครงสร้างไฟล์

ผู้ใช้เปิดเว็บไซต์หรือแอป Android เพื่อใช้งานส่วนติดต่อที่สร้างจาก Next.js และ React หน้าจอใช้ Firebase Authentication ระบุตัวตน ใช้ Firestore อ่านและเขียนข้อมูลตาม Rules และเรียก Cloud Functions สำหรับงานที่แยกไปทำงานเซิร์ฟเวอร์ เช่น การจองเลขออเดอร์และการจบออเดอร์

ไฟล์ที่เผยแพร่บน Firebase Hosting เป็นผล build ใน out ส่วนฟังก์ชันอยู่ใน functions/index.js และไฟล์สนับสนุน จึงต้องตรวจทั้งเว็บ ฐานข้อมูล และ Functions เมื่ออัปเดตระบบ การเผยแพร่เว็บอย่างเดียวไม่ทำให้โค้ด backend หรือ Rules เปลี่ยนตามโดยอัตโนมัติ

ตำแหน่ง	หน้าที่	ความสัมพันธ์
src/app	หน้าและ layout	ประกอบส่วนติดต่อผู้ใช้
src/components	หน้าร้าน ตะกร้า แชท dashboard	ใช้ข้อมูลจาก Context และ Firebase
src/context	AuthContext และ CartContext	สถานะร่วมของบัญชีและตะกร้า
src/lib	Firebase เลขออเดอร์ GPS สรุปยอด	ฟังก์ชันที่เรียกใช้ซ้ำ
src/types	ชนิดข้อมูล	ข้อตกลงข้อมูลใน TypeScript
functions	ตรรกะบนเซิร์ฟเวอร์	ตรวจสอบสิทธิ์และจัดการธุรกรรม
firestore.rules	ข้อกำหนดเข้าถึงฐานข้อมูล	ป้องกันค่าขอที่ไม่ผ่านเงื่อนไข
tests	ชุดทดสอบ	ตรวจพฤติกรรมและกรณีผิดพลาด

ขอบเขตความเชื่อถืออยู่ระหว่างเครื่องผู้ใช้กับบริการ backend ค่าในตะกร้าและ localStorage ช่วยแสดงหน้าจอได้ แต่ไม่ควรใช้เป็นหลักฐานสุดท้ายของราคาที่ต้องเรียกเก็บ รายงานจึงแยกสถาปัตยกรรมที่มีอยู่กับแนวทางย้ายการคำนวณราคาไป server ในส่วนปรับปรุง

3.6 แบบจำลองข้อมูลและความสัมพันธ์

Firestore จัดข้อมูลเป็น collection และ document ความสัมพันธ์ของโครงการใช้รหัสอ้างอิง เช่น shop_id และ customer_uid ไม่ใช่ foreign key ที่ฐานข้อมูลเชิงสัมพันธ์บังคับให้อัตโนมัติ ผู้พัฒนาต้องกำหนดเงื่อนไขความสอดคล้องของรหัสเหล่านี้เอง

กลุ่มข้อมูล	รหัสและช่องหลัก	ความสัมพันธ์
users	uid, display_name, phone	ผู้ใช้หนึ่งคนมีหลายออเดอร์
access	role, shop_id	สิทธิ์ของ uid และร้านที่ดูแล
shops	name, is_open, owner_uid	หนึ่งร้านมีหลายเมนูและหลายออเดอร์
menu_items	shop_id, name, price, options	เมนูเป็นของร้านหนึ่งแห่ง
orders	customer_uid, shop_id, items, status	ออเดอร์หนึ่งรายการผูกผู้ซื้อกับร้านหนึ่งแห่ง
orders/{id}/messages	sender_uid, text, created_at	หลายข้อความอยู่ใต้หนึ่งออเดอร์
credit_transactions	shop_id, order_id, amount, type	บัญชีเครดิตและค่าบริการของร้าน
system_settings	gp_enabled, gp_percent	การตั้งค่าที่ใช้ประกอบสถิติธุรกิจ

ตะกร้าอาจมีสินค้าจากหลายร้าน แต่เมื่อตรวจ checkout จะพบการวนสร้างออเดอร์แยกตามร้าน โดยใช้ group_id เชื่อมกลุ่มคำสั่งซื้อ

จึงไม่ควรอธิบายว่าแต่ละร้านเห็นรายการทุกอย่างในตะกร้าของลูกค้า

ร้านควรเห็นข้อมูลที่เกี่ยวข้องกับออเดอร์ของร้านตนเองเท่านั้น

รายการในออเดอร์เก็บชื่อและราคาที่ใช้ขณะสั่งไว้ด้วย

เพื่ออ่านประวัติได้แม้ข้อมูลเมนูภายหลังเปลี่ยน

การบันทึกภาพข้อมูลขณะสั่งมีประโยชน์ด้านประวัติ

แต่ต้องตรวจสอบความถูกต้องของข้อมูลก่อนบันทึก ไม่ใช่เชื่อราคาที่ลูกค้าส่งมาโดยตรง

3.7 พจนานุกรมข้อมูลออเดอร์

ชื่อช่อง	ชนิดโดยแนวคิด	ความหมาย
id / order_number	ข้อความ / ตัวเลข	รหัสเอกสารและเลขแสดงผล
group_id	ข้อความ	เชื่อมออเดอร์จากตะกร้ารอบเดียวกัน
shop_id / customer_uid	ข้อความ	ร้านผู้รับงานและเจ้าของออเดอร์
items	รายการของ OrderItem	เมนู จำนวน ตัวเลือก และหมายเหตุ
food_subtotal	จำนวนเงิน	ยอดสินค้าไม่รวมค่าส่ง
delivery_fee	จำนวนเงิน	ค่าส่งของร้านในออเดอร์นี้
total_amount	จำนวนเงิน	ยอดอาหารรวมค่าส่ง
gp_amount	จำนวนเงิน	ค่าบริการที่เกี่ยวข้องกับออเดอร์
payment_method	cash หรือ scan_merchant	เงินสดหรือสแกนจ่ายร้าน
status	ค่าที่กำหนดไว้	pending, cooking, delivering, completed, cancelled
created_at / completed_at	เวลา	เวลาสร้างและเวลาจบงาน
location / delivery_address	พิกัด / ข้อความ	จุดและรายละเอียดการจัดส่ง

ข้อมูลชนิด any ที่ใช้กับเวลาบางช่องสะท้อนว่าหน้าจ้ออาจรับทั้ง Firestore Timestamp และรูปแบบอื่น ผู้พัฒนาจึงต้องแปลงเวลาอย่างระวังในรายงานยอดขาย ตัวอย่าง summarizeSales ใช้วันที่ตาม Asia/Bangkok และเลือก completed_at ก่อน created_at สำหรับออเดอร์ที่จบแล้ว

3.8 เส้นทางคำสั่งซื้อและสถานะงาน

ขั้นที่ 1 ลูกค้าเลือกหน้าร้านและเมนู ระบุจำนวน ตัวเลือก และหมายเหตุ ระบบเก็บข้อมูลในตะกร้า ขั้นที่ 2 ตะกร้าจัดกลุ่มตามร้านและแสดงยอดเพื่อให้ตรวจทาน ขั้นที่ 3 เมื่อเข้าสู่ระบบแล้ว ลูกค้าระบุชื่อ เบอร์โทร ที่อยู่ และวิธีชำระเงินก่อนยืนยัน

ขั้นที่ 4 ในโค้ดฉบับอัปเดต checkout เรียก checkoutOrder พร้อม idempotency_key และรายการเมนูแยกร้าน backend อ่านข้อมูลร้านและเมนู คำนวณยอด และบันทึกผ่าน transaction ขั้นที่ 5 คืนรายการที่สร้างให้หน้าจอตีตาม ขั้นที่ 6 ร้านรับงาน ทำอาหาร จัดส่ง รับเงิน และปิดงาน ลูกค้าติดต่อร้านผ่านแชทของออเดอร์นั้นได้

เวอร์ชันเริ่มต้นสร้างออเดอร์จาก client แยกในรูป จึงมีความเสี่ยงบันทึกสำเร็จเพียงบางร้าน โค้ดฉบับอัปเดตย้ายไป transaction ใน functions/checkout.js และบันทึกค่าขอใน _checkout_requests เพื่อใช้คืนรายการเดิมเมื่อส่งซ้ำ อย่างไรก็ตาม ต้องตรวจกรณีค่าขอพร้อมกันและข้อมูลที่เปลี่ยนระหว่างการลองใหม่เพิ่มเติม

สถานะ	ความหมายต่อผู้ใช้	การเปลี่ยนที่พบใน Rules
pending	รอร้านรับงาน	ร้านรับเป็น cooking หรือยกเลิก
cooking	ร้านกำลังเตรียม	เปลี่ยนเป็น delivering หรือยกเลิก
delivering	ร้านกำลังส่ง	เรียก completeOrder เพื่อจบงาน
completed	ปิดงานแล้ว	ใช้ประกอบประวัติและสรุปยอด
cancelled	ยกเลิกรายการ	แยกออกจากยอดงานสำเร็จ

โค้ดฉบับอัปเดตไม่ให้ร้านเปลี่ยน completed ผ่าน direct update ใน Rules แต่ให้เรียก completeOrder ซึ่งรับสถานะ cooking หรือ delivering ก่อนจบงาน

ผู้ดูแลยังมีสิทธิ์ระดับสูงตาม Rules

จึงควรกำหนดขั้นตอนปฏิบัติของแอดมินให้รักษาความสอดคล้องของ ledger

3.9 ตะกร้า ตัวเลือก และตัวอย่างการคำนวณ

CartContext แยกรายการด้วยเมนู หมายเหตุ และชุดตัวเลือก

เมื่อผู้ใช้เพิ่มรายการที่มีองค์ประกอบเหมือนกัน ระบบเพิ่มจำนวนในรายการเดิม

แต่เมื่อหมายเหตุหรือตัวเลือกต่างกัน จะสร้างบรรทัดใหม่

วิธีนี้ช่วยไม่ให้ซ้ำซ้อนเดียวกันที่มีความต้องการต่างกันถูกรวมจนร้านอ่านผิด

ราคาต่อหน่วยเกิดจากราคาเมนูบวกผลรวมราคาตัวเลือก จากนั้นคูณจำนวนเพื่อเป็นยอดรายการ

และรวมเป็นยอดอาหารของแต่ละร้าน ในโค้ดตะกร้าที่ตรวจ ค่าส่งเป็น 10 บาทต่อร้าน

เมื่อตะกร้ามีสองร้านจึงคิดค่าส่งรวม 20 บาท ไม่ใช่เหมาจ่าย 10 บาททั้งตะกร้า

รายการสมมติ	วิธีคำนวณ	ยอดบาท
ร้าน ก ข้าวและไข่ 2 ชุด	$(50 + 10) \times 2$	120
ร้าน ข เครื่องดื่ม 1 แก้ว	30×1	30
ยอดอาหารรวม	$120 + 30$	150
ค่าส่งสองร้าน	10×2	20
ลูกค้ายจ่ายรวม	$150 + 20$	170
GP ร้าน ก เมื่อเปิด 5%	120×0.05	6.00
GP ร้าน ข เมื่อเปิด 5%	30×0.05	1.50

GP เป็นค่าบริการฝั่งร้าน ไม่บวกลูกค้าเพิ่มในตัวอย่างนี้ ฉบับอัปเดตให้ backend อ่าน

gp_enabled, gp_percent และ delivery_fee จาก system_settings/general

ขณะที่หน้าตะกร้าที่ตรวจยังคงค่าส่ง 10 บาทและ gpAmount 5% ในโค้ด

จึงต้องตรวจให้ยอดก่อนยืนยันกับยอดที่ backend บันทึกตรงกันเมื่อเปลี่ยนการตั้งค่า

3.10 บัญชีผู้ใช้และการกำหนดสิทธิ์

AuthContext ติดตามการเปลี่ยนบัญชีผ่าน Firebase Auth และโหลดข้อมูลผู้ใช้กับเอกสารสิทธิ์จากฐานข้อมูล แยกข้อมูลโปรไฟล์ออกจากข้อมูลอำนาจในการทำรายการ การแก้ไขหรือรูปของตนเองจึงไม่ควรทำให้ได้รับสิทธิ์ร้านหรือแอดมินเพิ่ม

Firestore Rules ใช้ collection access เป็นแหล่งสิทธิ์หลัก admin ตรวจ role เป็น admin ส่วน merchant ตรวจทั้ง role และ shop_id การตรวจสอบสองส่วนทำให้ระบบแยกระหว่างการเป็นร้านค้าและการเป็นเจ้าของร้านที่กำลังร้องขอข้อมูล

สำหรับออเดอร์ ฟังก์ชัน participant ยอมให้ลูกค้าเจ้าของรายการ ร้านของออเดอร์ หรือแอดมินเข้าถึง ข้อความใต้ orders/{id}/messages จึงใช้เงื่อนไขผู้เกี่ยวข้องของออเดอร์แม่ เพื่อไม่ให้บุคคลทั่วไปอ่านบทสนทนาและรายละเอียดการส่งของได้

บันทึกตรวจสอบในระยะแรกพบการ fallback สิทธิ์เก่า ส่วนโค้ด backend ที่อัปเดตใช้ตัวช่วยตรวจสอบ access แล้ว การปรับปรุงนี้ทำให้แนวทางกำหนดบทบาทสอดคล้องขึ้น แต่ยังควรทดสอบบัญชีที่ถูกถอนสิทธิ์และทุก endpoint เพื่อยืนยันผลจริง

การซ่อนปุ่มในหน้าจอช่วยให้ใช้งานง่าย แต่ไม่ใช่การรักษาสิทธิ์ที่เพียงพอ ผู้ใช้สามารถสร้างค่าขอเองได้ ระบบจึงต้องปฏิเสธที่ฐานข้อมูลหรือ backend แม้ค่าขอนั้นไม่เคยผ่านหน้าจอปกติ นี่เป็นเหตุผลที่ต้องมีชุดทดสอบ Rules แยกจากการกดหน้าจอ

3.11 แชน แชท ตำแหน่ง และการแจ้งเตือน

แชทเก็บเป็นข้อความย่อยได้แต่ละออเดอร์ ข้อความระบุผู้ส่ง บทบาท เวลา และสถานะอ่านแล้ว การผูกแชทกับออเดอร์ช่วยให้ทั้งสองฝ่ายรู้ว่ากำลังคุยเรื่องรายการใด เช่น ขอเปลี่ยนจุดรับหรือยืนยันว่าอาหารมาถึงแล้ว โดยไม่ต้องย้อนหาเมนูจากบทสนทนาที่ไม่เกี่ยวข้อง

ChatModal รองรับข้อความและรูปภาพ มีการตรวจประเภทไฟล์ก่อนบีบอัดรูป ฝั่งหน้าจอลดขนาดภาพด้านยาวไม่เกิน 800 พิกเซลและแปลงเป็น JPEG ด้วยค่า 0.76 ตามโค้ดที่ตรวจ ขนาดไฟล์ที่ได้ขึ้นอยู่กับเนื้อหาภาพ ไม่ควรรับรองว่าจะมีขนาดตายตัวทุกภาพ

ตำแหน่งจัดส่งใช้ข้อมูลละติจูดและลองจิจูดรวมกับคำอธิบายที่อยู่ การขอ GPS บน Android และเว็บใช้คนละเส้นทางตามสภาพแวดล้อม

หากผู้ใช้ไม่อนุญาตควรแสดงข้อความแก้ปัญหาที่เหมาะสม ไม่ถือว่าเกิดความผิดพลาดของผู้ใช้ และไม่ควรทำให้การปฏิเสธสิทธิ์ถูกข้ามอย่างเจียบ ๆ

ระบบมีโค้ดเกี่ยวกับเสียงและ push notification รวมทั้ง badge งานร้านค้า การแจ้งเตือนเป็นส่วนช่วยให้ร้านรับรู้เหตุการณ์ แต่ต้องตรวจแยกจากการบันทึกออเดอร์ บันทึกความพร้อมระยะแรกระบุว่า endpoint บางตัวตอบ 403 ก่อนถึง handler จึงยังไม่สามารถอ้างว่าแจ้งเตือนทุกเส้นทางบน production ได้แล้ว

การตรวจใช้งานจริงควรครอบคลุมแอปเปิดอยู่ เบื้องหลัง ปิดจอ อินเทอร์เน็ตหลุดแล้วกลับมา และการเปิดออเดอร์จากการแจ้งเตือน แต่ละกรณีอาจมีเงื่อนไขสิทธิ์หรือข้อจำกัดอุปกรณ์ต่างกัน

3.12 การจบออเดอร์และการสรุปยอดขาย

completeOrder ตรวจสอบว่ามีบัญชี ตรวจสอบรหัสออเดอร์

และตรวจสอบว่าผู้เรียกเป็นร้านที่เกี่ยวข้องหรือแอดมินก่อนทำงาน ภายใน transaction อ่านออเดอร์ร้าน การตั้งค่า และเครื่องหมายการประมวลผล ก่อนเขียนสถานะ completed

ปรับตัวนับยอดขายและเครดิต รวมถึงบันทึก credit_transactions เมื่อมีการหักค่าบริการ

หากออเดอร์ completed อยู่แล้ว ฟังก์ชันคืนผลว่าจบแล้ว การตรวจสอบสถานะซ้ำภายใน

transaction ช่วยดูแลกรณีมีค่าขอพร้อมกัน เครื่องหมาย _completed_orders

ใช้กันการนับยอดขายซ้ำอีกชั้นหนึ่ง อย่างไรก็ตาม

ความปลอดภัยของทั้งระบบยังขึ้นกับการป้องกันช่องทางที่เขียนฐานข้อมูลโดยไม่ผ่านฟังก์ชันนี้ด้วย

summarizeSales นับเฉพาะออเดอร์ completed ในช่วงวันที่ที่เลือก และอ่านยอด GP

จากธุรกรรมที่หักจริง แทนการนำยอดขายทั้งหมดคูณอัตราคงที่ วิธีนี้ช่วยให้รายงานตรงกับ

ledger ในกรณีเปิดหรือปิดค่าบริการ และแยกยอดรับจากลูกค้าออกจากค่าบริการแพลตฟอร์ม

ตัวเลข	ความหมาย	ข้อควรระวัง
food	ยอดอาหารของออเดอร์สำเร็จ	ไม่รวมรายการยกเลิก
delivery	รายรับค่าส่งในงานสำเร็จ	ไม่ใช่รายได้ของแพลตฟอร์มโดยอัตโนมัติ
collected	ยอดรวมที่ออเดอร์ระบุ	ไม่ได้เป็นหลักฐานโอนเงินจริงจากธนาคาร
gp	ค่าบริการจาก ledger	แยกจากอัตราที่แสดงใน settings
net	collected ลบ GP	ยังไม่ได้หักต้นทุนอาหารและค่าใช้จ่ายร้าน

3.13 การติดตั้งและเตรียมสภาพแวดล้อม

ขั้นที่ 1 เตรียมสำเนา repository และอ่าน AGENTS.md กับ README ของโครงการ
ตรวจเวอร์ชัน runtime และคู่มือ Next.js ใน node_modules/next/dist/docs
ให้ตรงรุ่นก่อนแก้โค้ด ขั้นที่ 2 ติดตั้ง dependency ฝั่งเว็บด้วย npm ci จากไฟล์เดอรัลหลัก
และเตรียม dependency ของ functions ตาม lockfile ที่มี หากไม่มี lockfile
ของส่วนใดต้องบันทึกเวอร์ชันที่ติดตั้งไว้

ขั้นที่ 3 ใช้ .env.local.example เป็นรายการชื่อค่าที่ต้องตั้ง
แล้วสร้างค่าท้องถิ่นให้ตรงโครงการของตน ไม่ใส่รหัสผ่าน private key หรือ token
จริงลงรายงานหรือ repository การตั้งค่าบริการ LINE และ Firebase
ต้องสัมพันธ์กับโดเมนและช่องทางของเจ้าของโครงการ

ขั้นที่ 4 รัน npm run dev เพื่อพัฒนาในเครื่อง ตรวจหน้าร้าน บัญชี
และข้อความผิดพลาดก่อนต่อขั้นตอนที่มีการเขียนข้อมูล ขั้นที่ 5
เตรียมฐานข้อมูลทดสอบและบัญชีแต่ละบทบาท หลีกเลี่ยงนำคำสั่ง seed หรือ reset
ไปใช้กับระบบจริงโดยไม่ตรวจเป้าหมาย

ขั้นที่ 6 ใช้ emulator สำหรับการทดสอบ Rules และการเข้าถึงข้ามบัญชี โปรเจกต์มี
firebase.emulators.json และสคริปต์ที่ระบุ demo-huaychan-security อยู่แล้ว
การใช้ข้อมูลจำลองทำให้ตรวจกรณีผิดพลาดได้โดยไม่กระทบยอดขายหรือบทสนทนาจริง

คำสั่งจาก package.json	จุดประสงค์
npm run dev	เปิดโหมดพัฒนา
npm run build	ตรวจและสร้างไฟล์ export
npm run test:functions	ทดสอบตรรกะ Functions
npm run test:sales	ทดสอบการสรุปยอด
npm run test:security	ทดสอบสิทธิ์ผ่าน emulator
npm run test:readiness	ทดสอบเงื่อนไขก่อนเปิดใช้

3.14 การเผยแพร่และดูแลหลังติดตั้ง

เมื่อผลตรวจที่จำเป็นผ่าน ให้ build เว็บและตรวจไฟล์ใน out ซึ่งตรงกับ hosting.public ของ firebase.json โครงการกำหนด Hosting สอง site จึงต้องระบุเป้าหมายให้ถูกในการเผยแพร่ และตรวจว่าโดเมนที่เปิดคือ huaychan.web.app การอัปเดต Functions และ Firestore Rules ต้องรวมอยู่ในแผนเผยแพร่เมื่อมีการแก้ไขฝั่ง server

ก่อนเผยแพร่ควรเก็บรหัสเวอร์ชันหรือ commit ที่ผ่านทดสอบ สำรองค่าที่ต้องใช้ และเตรียมวิธีย้อนกลับ เมื่อเผยแพร่แล้วตรวจสอบเส้นทางสำคัญด้วยบัญชีทดสอบที่กำหนด ได้แก่ ลูกค้าส่งรายการ ร้านรับงาน และแอดมินตรวจรายการ โดยแยกข้อมูลทดสอบออกจากข้อมูลใช้งานจริง

ฝั่ง Android สคริปต์ build:android ใช้ next build และ cap sync android การสร้างไฟล์ APK ได้ยังไม่เท่ากับผ่านการรับรองบนอุปกรณ์จริง บันทึก readiness ระยะแรกระบุว่าต้องเตรียม signing configuration เพิ่ม versionCode และตรวจการอัปเดตที่รุ่นเดิมก่อนแจกเป็นรุ่นพร้อมใช้งาน

การดูแลหลังใช้งานควรติดตามคำขอผิดพลาด การแจ้งเตือนที่ไม่ถึงร้าน รายการซ้ำ และความไม่ตรงกันระหว่างออเดอร์กับ ledger ตั้งช่องทางรับแจ้งปัญหาและบันทึกรหัสออเดอร์ที่เกี่ยวข้อง เพื่อให้ผู้ดูแลสืบค้นได้โดยไม่ต้องขอข้อมูลส่วนตัวเกินจำเป็น

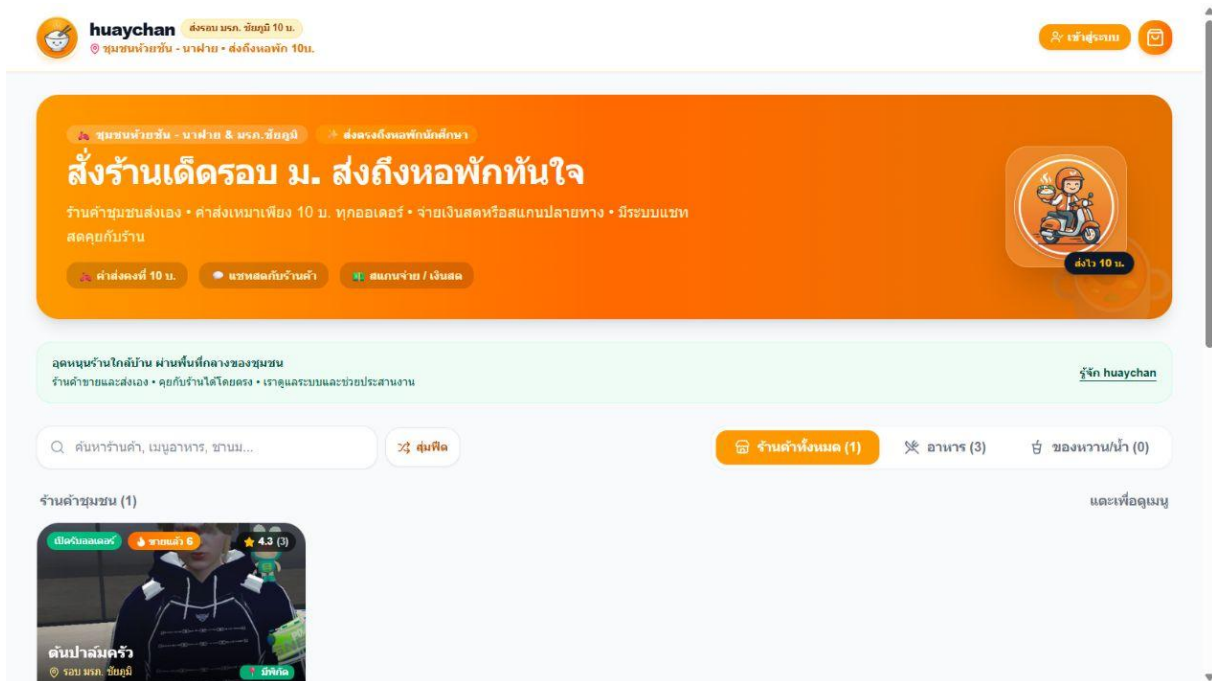
ระบบควรกำหนดวงจรการเก็บข้อมูลอย่างชัดเจน เช่น ระยะเวลาที่จำเป็นสำหรับข้อความ รูปภาพ และประวัติคำสั่งซื้อ การล้างข้อมูลควรเป็นขั้นตอนเฉพาะที่ตรวจสอบได้ การล้างแบบไม่ควรถ้าให้หลักฐานยอดขายหรือธุรกรรมที่ต้องเก็บกฎหมาย รายงานนี้ไม่ได้เรียกใช้คำสั่งล้างข้อมูลหรือเผยแพร่ระบบ

บทที่ 4 ผลการพัฒนาและการทดสอบ

4.1 ผลการตรวจหน้าสาธารณะ เว็บไซต์ huaychan.web.app แสดงแนวคิดร้านค้าชุมชนตนเอง มีองค์ค้นหา หมวดร้านและสินค้า ปุ่มเข้าสู่ระบบ และตะกร้า

ส่วนเรื่องเล่าระบบบทบาทร้านกับแพลตฟอร์มสอดคล้องกับโจทย์ของผู้จัดทำ

ภาพด้านล่างบันทึกจากหน้าเว็บจริงในวันที่ 17 กันยายน 2569



ภาพที่ 1 หน้าเว็บไซต์สาธารณะ huaychan ณ วันที่ตรวจ

ภาพนี้ยืนยันการแสดงผลหน้าสาธารณะในเวลาที่เราตรวจเท่านั้น จำนวนร้าน คะแนน และยอดขายที่ปรากฏไม่ถูกนำมาสรุปเป็นผลการทดลองหรือสถิติการใช้งานโครงการ และการเปิดหน้านี้ไม่ได้ยืนยันว่าการชำระเงินหรือการจัดส่งเกิดขึ้นจริง

4.2 ผลการตรวจองค์ประกอบระบบ

ส่วนงาน	หลักฐานที่พบ	ระดับการยืนยันในรายงาน
หน้าร้านและแนวคิด	เว็บไซต์จริงและ src/app/page.tsx	เปิดดูหน้าสาธารณะแล้ว
บัญชีและสิทธิ์	AuthContext และ firestore.rules	ตรวจสอบซอร์สโค้ด
ตะกร้าหลายร้าน	CartContext และ CartCheckoutModal	ตรวจสอบวิธีจัดกลุ่มและคำนวณ
แชทออเดอร์	ChatModal และ Rules ของ messages	ตรวจสอบโครงสร้างและเงื่อนไข
จบออเดอร์และ GP	completeOrder และ unit tests	ตรวจสอบโค้ดและรันทดสอบย่อย
รายงานยอดขาย	salesSummary และ tests	ตรวจสอบโค้ดและรันทดสอบย่อย
GPS	geolocation และ tests	ทดสอบตรรกะจำลอง
Android และ push	capacitor.config.ts และ android	พบโครงสร้าง ยังไม่ทดสอบเครื่องจริงรอบนี้

ผลการพัฒนาควรนำเสนอพร้อมวันที่และเวอร์ชัน ระหว่างจัดทำเอกสารมีการอัปเดตโครงการเพิ่ม backend checkout และ idempotency รายงานจึงเก็บข้อค้นพบระยะแรกเป็นประวัติ และระบุการเปลี่ยนแปลงล่าสุดในหัวข้อถัดไป
ผลตรวจจากบันทึกโครงการต้องแยกจากการทดสอบที่รันเองในงานรายงาน

ตัวอย่างคือ notifyNewOrder ซึ่งเคยมีรายงาน 403 บันทึกโครงการฉบับอัปเดตระบุว่าแก้ IAM และเผยแพร่แล้ว แต่การส่งถึงมือถือจริงยังต้องตรวจ acceptance test ไม่ควรสรุปจากการเข้าถึง endpoint อย่างเดียวว่าแจ้งเตือนถึงผู้รับทุกครั้ง

4.3 ผลชุดทดสอบที่รันในการจัดทำรายงาน

วันที่ตรวจรันชุดทดสอบด้วย Node test runner โดยระบุไฟล์ badge.test.cjs, line-identity.test.cjs, complete-order.test.cjs, sales-summary.test.mjs และ geolocation.test.cjs ผลรวมผ่าน 20 จาก 20 กรณี ไม่มีกรณีล้มเหลว ชุดทดสอบเหล่านี้ไม่ได้สร้างออเดอรับระบบจริง

ชุด	จำนวนผ่าน	พฤติกรรมสำคัญที่ตรวจ
Badge	5	นับงานรอรับ อัปเดตเมื่อเปลี่ยนและแบ่งส่ง token
LINE identity	3	ใช้ตัวตนที่ตรวจสอบและปฏิเสธ token ผิด
Complete order	6	จบงาน หัก GP ไม่ซ้ำ และปฏิเสธผู้ไม่มีสิทธิ์
Geolocation	4	ขอสิทธิ์ พิกัดศูนย์ และจัดการข้อผิดพลาด
Sales summary	2	วันไทย ยอดหักจริง และช่วงข้อมูลว่าง
รวม	20	ผ่านตามขอบเขตของแต่ละชุด

การตีความผล: หลักฐานสนับสนุนความถูกต้องของตรรกะในกรณีที่จำลองไว้ ตัวอย่าง completeOrder ตรวจการเรียกซ้ำแล้วไม่หักค่าบริการซ้ำภายในเส้นทางฟังก์ชัน แต่ไม่ได้พิสูจน์ว่าไม่มีช่องทางอื่นเขียนสถานะจบงานโดยตรง และไม่ได้ยืนยัน IAM ของ endpoint ที่ deploy จริง

ข้อจำกัดของการทดสอบครั้งนี้คือไม่ได้รัน emulator suite ใหม่ทั้งหมด ไม่ได้ build ใหม่เพื่อรายงาน และไม่ได้ทดสอบความพึงพอใจหรือประสิทธิภาพกับผู้ใช้จริง ผลด้าน build, emulator และ Android ในหัวข้อถัดไปจึงอ้างบันทึกเดิมอย่างชัดเจน

4.4 ประเด็นจากบันทึกตรวจสอบความพร้อม

ตารางต่อไปเป็นประวัติข้อค้นพบระยะแรกจาก LAUNCH_READINESS_REVIEW.md วันที่ 17 กันยายน 2026 ก่อนการแก้ระหว่างจัดทำรายงาน ไม่ใช่สถานะล่าสุดของทุกข้อ
บันทึกฉบับอัปเดตระบุว่าทำ backend checkout แก่ IAM และสร้าง signed APK แล้ว
โดยยังต้องทดสอบ acceptance บนอุปกรณ์จริงก่อน pilot

ประเด็น	ผลหรือข้อค้นพบในบันทึก	แนวทางแก้
ราคาจาก client	ส่งราคาต่ำกว่าราคาเมนูได้ใน emulator	ให้ server อ่านเมนูและคำนวณเอง
จบงานข้าม GP	direct update ไป completed ได้	บังคับจบผ่าน backend
การสั่งซ้ำ	เอกสารต่าง id ใช้ group/number เดิมได้	มี request key และ idempotency
Functions บางตัว	notifyNewOrder และ topUp ตอบ 403	แก้ invocation และทดสอบด้วยบัญชี
สิทธิ์ร้าน	security suite เดิมผ่าน 15/16	เพิ่ม fixture และกำหนดกรณี settings หาย
Android	lint ผ่าน แต่ยังไม่ตรวจเครื่องจริง	ทดสอบอุปกรณ์และ signed release

บันทึกฉบับอัปเดตระบุว่า build ผ่าน และ Functions tests ผ่าน 20/20 รวม checkout ใหม่ ส่วนผลที่ผู้เรียบเรียงรันเองก่อนอัปเดตคือ 20 กรณีจาก Functions เดิมรวม GPS และ sales ซึ่งเป็นคนละชุด ห้ามนำตัวเลข 20/20 ทั้งสองชุดมาแทนกัน ข้อมูล production และ APK ในบันทึกเป็นหลักฐานที่รายงานต่อจากโครงการ ไม่ได้ตรวจซ้ำครบในงานเอกสาร

รายการข้อจำกัดเป็นข้อมูลเพื่ออธิบายความก้าวหน้าและแผนปรับปรุงได้ในการสอบโครงการ ไม่ควรลบออกเพื่อทำให้รายงานดูสมบูรณ์เกินจริง
เมื่อแก้แล้วต้องแนบผลทดสอบและวันที่ใหม่แทนการเปลี่ยนคำว่าผ่านโดยไม่มีหลักฐาน

4.5 แผนทดสอบระบบครบเส้นทาง

กรณี	ขั้นตอนย่อ	ผลที่คาดหวัง
หนึ่งร้าน	เลือกเมนู ตรวจสอบยอด ยืนยัน รับงาน จบงาน	รายการและ ledger ตรงกัน
หลายร้าน	สั่งสองร้านและเปิดบัญชีร้านแต่ละแห่ง	แต่ละร้านเห็นเฉพาะงานตน
กดซ้ำ	ส่งค่าขอเดิมซ้ำด้วย request key เดิม	คีนออเดอร์เดิมไม่สร้างซ้ำ
ราคาปลอม	ส่งยอดต่ำกว่าราคาเมนู	server ปฏิเสธหรือคำนวณใหม่
ร้านปิด	สั่งหลังร้านปิดรับ	ไม่สร้างงานที่ร้านรับไม่ได้
ข้ามบัญชี	อ่านออเดอร์หรือแชทคนอื่น	ปฏิเสธการเข้าถึง
เน็ตหลุดบางช่วง	ตัดเครือข่ายระหว่างยืนยัน	แสดงผลขัดและลองใหม่ไม่ซ้ำ
GP เปิดและปิด	จบรายการในแต่ละค่า settings	ledger ตรงกติกาที่กำหนด
แจ้งเตือนมือถือ	ทดลองหน้าแอปและเบื้องหลัง	ได้รับและเปิดงานถูกออเดอร์

สถานะของตารางนี้คือแผนตรวจรับที่เสนอ ผู้ทดสอบควรบันทึกเวอร์ชันเว็บ รุ่น Android บัญชีทดสอบ ข้อมูลนำเข้า ผลจริง เวลา และภาพหลักฐาน โดยไม่ใส่รหัสผ่านลงแบบบันทึก แต่ละกรณีควรเริ่มจากข้อมูลที่อยู่สภาพและตรวจผลในฐานข้อมูลควบคู่กับหน้าจอ

บทที่ 5 สรุป อภิปรายผล และแนวทางพัฒนา

5.1 สรุปโครงการ huaychan แสดงแนวทางพัฒนาระบบจากข้อสังเกตในชุมชน โดยรวบรวมร้านและเมนูให้ค้นหาได้ และเชื่อมการเลือกสินค้ากับออเดอร์และแชท โครงสร้างมีลูกค้า ร้านค้า และผู้ดูแลระบบ ใช้เว็บเป็นฐานและมีส่วนเชื่อม Android รองรับงานร้านค้าและความสามารถบนมือถือ

5.2 อภิปรายผล จุดที่ตอบโจทย์คือข้อมูลถูกจัดเป็นโครงสร้างเดียวกัน เช่น เมนูมีราคา ออเดอร์มีรายการและสถานะ และแชทมีออเดอร์อ้างอิง อย่างไรก็ตาม คุณค่าที่คาดหวังต่อร้านและลูกค้าต้องแยกจากผลเชิงตัวเลขที่ยังไม่ได้เก็บ จึงกล่าวได้ว่าระบบรองรับแนวทางนี้ แต่ยังสรุปไม่ได้ว่าเพิ่มรายได้หรือลดเวลาซื้อได้กี่เปอร์เซ็นต์

5.3 ข้อจำกัด ฉบับอัปเดตมีการสร้างออเดอร์ฝั่ง backend และบันทึก idempotency แล้ว แต่ยังคงทดสอบการส่งพร้อมกัน การตรวจตัวเลือกอาหาร ความตรงกันของยอดหน้าจอกับยอด server และงานจริงบน Android การมีโค้ดแก้ไขไม่ได้แทนผลทดสอบครบเส้นทาง

5.4 ลำดับปรับปรุง ตรวจ backend checkout และการกันซ้ำให้ครบกรณีผิดพลาด ยืนยันว่าการจบงานกับ ledger สอดคล้องและไม่มีสิทธิ์เกาหลงเหลือ จากนั้นตรวจ production ด้วยบัญชีทดสอบและอุปกรณ์ Android จริงก่อนทดลองใช้กลุ่มเล็ก

5.5 บทเรียน โครงการชุมชนไม่ได้มีเพียงการสร้างหน้าจอ ต้องแปลงวิธีทำงานจริงเป็นข้อมูล กติกา และสิทธิ์ที่ตรวจสอบได้

การอธิบายข้อจำกัดพร้อมหลักฐานและแนวทางแก้ไขแสดงความเข้าใจระบบได้ดีกว่าการอ้างว่า ทุกส่วนสมบูรณ์แล้ว

5.6 การประเมินกับชุมชนและการต่อยอด

หลังปิดประเด็นที่มีผลต่อความถูกต้องของออเดอร์

ควรเชิญผู้ทดลองที่ยินยอมจากทั้งลูกค้าและร้านค้า โดยกำหนดจำนวนและวิธีคัดเลือกก่อนเริ่ม
บันทึกงานที่ให้ทำเหมือนกัน เช่น ค้นหาร้าน ส่งเมนูหนึ่งรายการ และส่งข้อความแจ้งจุดรับ
จากนั้นสังเกตว่าทำสำเร็จหรือไม่ ใช้เวลาเท่าใด และติดขัดตรงไหน

แบบสอบถามควรแยกความชัดเจนของเมนู ความเข้าใจโดยรวม การติดตามสถานะ
และความสะดวกในการติดต่อ ไม่ถามรวมเพียงว่าชอบแอปหรือไม่ ข้อมูลเชิงคุณภาพ เช่น
ผู้ใช้สับสนคำสั่งหลายร้าน มีประโยชน์ต่อการแก้หน้าจอ แม้จำนวนผู้ทดลองยังไม่มาก

การสรุปผลต้องรายงานจำนวนผู้ตอบจริง วิธีเก็บข้อมูล และข้อจำกัดของกลุ่มตัวอย่าง
หากมีการวัดเวลาก่อนและหลังควรใช้โจทย์ใกล้เคียงกัน
และอธิบายว่าทักษะผู้ใช้หรือความเร็วอินเทอร์เน็ตอาจมีผล
ห้ามเติมคะแนนความพึงพอใจหรือจำนวนผู้ทดลองโดยไม่มีข้อมูล

แนวทางต่อยอดที่สัมพันธ์กับโจทย์ ได้แก่ เขตจัดส่งที่กำหนดชัด
ปรับขั้นตอนร้านรับออเดอร์ให้เหมาะกับมือถือ จัดการรูปภาพแยกจากเอกสารเมื่อข้อมูลเพิ่ม
และเพิ่มการติดตามข้อผิดพลาด
การเพิ่มร้านควรทำหลังขั้นตอนเดิมมีความถูกต้องและผู้ดูแลสามารถช่วยเหลือร้านได้

หลักการของโครงการยังคงเป็นการช่วยให้ร้านเล็กมีพื้นที่แสดงสินค้า
และให้ลูกค้าติดต่อคนขายโดยตรง

การขยายระบบจึงควรประเมินผลต่อวิธีการทำงานของร้านและความเข้าใจของลูกค้า
ไม่เพิ่มความสามารถเพียงเพราะเทคโนโลยีทำได้

- [1] Firebase. Get realtime updates with Cloud Firestore.
<https://firebase.google.com/docs/firestore/query-data/listen> สืบค้น 17 กันยายน 2569
 - [2] Firebase. Transactions and batched writes.
<https://firebase.google.com/docs/firestore/manage-data/transactions> สืบค้น 17 กันยายน 2569
 - [3] LINE Developers. Using user data in LIFF apps and servers.
<https://developers.line.biz/en/docs/liiff/using-user-profile/> สืบค้น 17 กันยายน 2569
 - [4] Capacitor. Capacitor Documentation. <https://capacitorjs.com/docs> สืบค้น 17 กันยายน 2569
 - [5] Next.js. How to create a static export of your Next.js application. คู่มือที่ติดตั้งใน node_modules/next/dist/docs/01-app/02-guides/static-exports.md ของโครงการรุ่น 16.3.5 ตรวจ 17 กันยายน 2569
 - [6] โครงการ huaychan. ซอร์สโค้ด package.json, src/types/index.ts, src/context, src/components, src/lib, functions/index.js, firestore.rules, firebase.json และ capacitor.config.ts ตรวจ 17 กันยายน 2569
 - [7] โครงการ huaychan. LAUNCH_READINESS_REVIEW.md. บันทึกตรวจสอบความพร้อม วันที่ 17 กันยายน 2026
 - [8] โครงการ huaychan. เว็บไซต์หน้าสาธารณะ <https://huaychan.web.app> ตรวจด้วยเบราว์เซอร์ 17 กันยายน 2569
- ข้อมูลที่มาของแนวคิดใช้คำอธิบายของผู้จัดทำที่ให้ประกอบการเรียบเรียงรายงาน
ยังไม่มีเอกสารโครงงานจากมหาวิทยาลัยไทยแนบมาเป็นแหล่งอ้างอิงเพิ่มเติม

- 1 เปิด huaychan.web.app ตรวจสอบว่าร้านและสินค้าที่ต้องการอยู่ในพื้นที่ให้บริการ ใช้ช่องค้นหาและหมวดสินค้าเพื่อช่วยหาเมนู ก่อนสั่งควรอ่านสถานะร้าน ราคา และรายละเอียดเมนูให้ครบ
- 2 เปิดเมนู เลือกตัวเลือกและจำนวน หากต้องการแยกความต้องการ เช่น ไม่ใส่พริกกับใส่พริก ให้ตรวจรายการในตะกร้าว่าไม่ถูกรวมผิด ตรวจสอบจำนวนร้านด้วย เพราะตะกร้าหลายร้านคิดค่าส่งตามร้านในโค้ดตะกร้าที่ตรวจ
- 3 เข้าสู่ระบบตามช่องทางที่รองรับ ระบุชื่อและเบอร์ที่ร้านติดต่อได้ กรอกที่อยู่พร้อมจุดสังเกต หากใช้แผนที่หรือ GPS ให้ตรวจว่าหมดตรงกับจุดรับ ไม่อาศัยพิกัดอย่างเดียวเมื่ออาคารอยู่ติดกัน
- 4 เลือกเงินสดหรือสแกนจ่ายร้านตามตัวเลือกที่แสดง ตรวจยอดอาหาร ค่าส่ง และยอดรวมอีกครั้งก่อนยืนยัน หากใช้เงินสดและต้องการเงินทอนให้กรอกหมายเหตุที่มีในหน้าจอ
- 5 หลังยืนยัน ให้ตรวจรหัสออเดอร์และสถานะ ตะกร้าที่มีหลายร้านอาจได้หลายออเดอร์ หากไม่แน่ใจว่าส่งสำเร็จหรือไม่ให้ตรวจรายการก่อนกดซ้ำ เพราะการกัณค่าขอซ้ำเป็นส่วนที่เพิ่งปรับปรุงและยังต้องตรวจรับ
- 6 ใช้แชทของออเดอร์เพื่อติดต่อร้านเมื่อจำเป็น เช่น แจ้งจุดนัดรับหรือสอบถามงาน หากเกิดปัญหาเรื่องอาหารและการส่งให้ติดต่อร้าน ส่วนเหตุขัดข้องของระบบสามารถแจ้งแอดมินให้ช่วยประสานงาน

ภาคผนวก ก คู่มือสำหรับร้านค้าและผู้ดูแล

ร้านค้า เข้าสู่ระบบด้วยบัญชีที่ได้รับสิทธิ์ร้าน ตรวจสอบร้าน เบอร์ติดต่อ ช่องทางรับเงิน และเมนูของตน ปรับราคา ตัวเลือก และความพร้อมของสินค้าให้ตรงกับที่ขายจริง เปิดรับออเดอร์เมื่อพร้อมให้บริการ และปิดเมื่อไม่สามารถรับงานเพิ่มได้

เมื่อมีรายการใหม่ ให้ตรวจจำนวนอาหาร หมายเหตุ เบอร์โทร และที่อยู่ก่อนรับงาน เปลี่ยนสถานะตามความคืบหน้าที่เกิดขึ้น หากต้องตกลงรายละเอียดเพิ่มให้ใช้แชทของออเดอร์ เมื่อส่งและรับเงินแล้วจึงปิดงานตามขั้นตอนของระบบ ตรวจสอบประวัติและยอดที่เกี่ยวข้องภายหลัง

การอ่านรายงานร้าน เลือกช่วงวันที่ ตรวจสอบยอดอาหาร ค่าส่ง และค่าบริการที่ถูกหักจริง ค่า net ในโปรแกรมเป็นยอดรับลบ GP ไม่ใช่กำไรสุทธิ เพราะยังไม่มีต้นทุนวัตถุดิบ ค่าแรง และค่าเดินทางอยู่ในการคำนวณนี้

ผู้ดูแล ตรวจสอบสิทธิ์ของบัญชีก่อนอนุญาตให้จัดการร้าน ติดตามปัญหาด้วยรหัสออเดอร์ ตรวจสอบข้อมูลที่เกี่ยวข้องตามสิทธิ์ และประสานคู่ซื้อขายเมื่อจำเป็น การเปลี่ยนการตั้งค่า GP หรือเครดิตควรมีบันทึกเหตุผลและผู้ทำรายการเพื่อให้ตรวจสอบย้อนกลับได้

ก่อนใช้งานจริง ตรวจสอบว่าฟังก์ชันแจ้งเตือนและเติมเครดิตเรียกได้ตามบทบาท รวมทั้งผลการหักค่าบริการ หากพบข้อผิดพลาดอย่าสรุปจากหน้าจออย่างเดียว ให้เทียบออเดอร์กับธุรกรรมและบันทึกเหตุการณ์ ไม่แก้ยอดย้อนหลังโดยไม่มีหลักฐาน

ภาคผนวก ข โค้ดตะกร้าและการจัดกลุ่มร้าน

ตัวอย่างต่อไปนี้อัดจากโค้ดจริง เลือกเฉพาะช่วงที่จำเป็นเพื่ออธิบายกลไก
ไม่ใช่ไฟล์ที่นำไปใช้งานได้

ที่มา src/context/CartContext.tsx บรรทัด 85 ถึง 109 ณ 17 กันยายน 2569

```
85 // Group items by shop
86 const groupedItems = useMemo(() => {
87   const groups: { [shopId: string]: ShopCartGroup } = {};
88   items.forEach((item) => {
89     const sId = item.shop_id || 'default_shop';
90     if (!groups[sId]) {
91       groups[sId] = {
92         shop: {
93           id: sId,
94           name: item.shop_name || 'ร้านค้าชุมชน',
95           phone: item.shop_phone || '',
96         },
97         items: [],
98         subtotal: 0,
99         deliveryFee: 10,
100        total: 10,
101      };
102    }
103    groups[sId].items.push(item);
104    const itemUnitPrice = item.unit_price ?? item.price;
105    groups[sId].subtotal += itemUnitPrice * item.quantity;
106    groups[sId].total = groups[sId].subtotal + groups[sId].deliveryFee;
107  });
108   return Object.values(groups);
109 }, [items]);
```

groups ใช้ shop_id เป็นกุญแจ เมื่อพบร้านใหม่จะสร้างกลุ่มและกำหนดค่าส่งเริ่มต้น 10 บาท
จากนั้นเพิ่มสินค้าของร้านเข้ากลุ่ม คำนวณยอดอาหารจาก unit_price หากไม่มีจึงใช้ price
และคูณจำนวนก่อนรวมยอด

ตัวอย่าง เมนูราคา 50 บาทมีตัวเลือกเพิ่ม 10 บาท จำนวน 2 ชุด ค่า unit_price เท่ากับ 60
และเพิ่ม subtotal 120 บาท รายการของร้านอื่นอยู่คนละกลุ่ม จึงได้ total ของร้านนี้ 130 บาท
โดยยังไม่รวมยอดของร้านอื่น

ประเด็นเรียนรู้คือ useMemo คำนวณใหม่เมื่อ items เปลี่ยน ช่วยให้ค่าจัดกลุ่มสัมพันธ์กับตะกร้า
แต่เป็นการคำนวณเพื่อหน้าจอนั่น

ความถูกต้องของราคาที่น่าไปบันทึกต้องยืนยันจากข้อมูลเมนูฝั่ง server อีกครั้ง

ที่มา src/lib/orderNumber.ts บรรทัด 1 ถึง 8 ณ 17 กันยายน 2569

```
1 import { httpsCallable } from 'firebase/functions';
2 import { functions } from './firebase';
3
4 export async function getNextOrderNumbers(count: number = 1): Promise<number[]> {
5   const reserve = httpsCallable<{ count: number }, { numbers: number[] }>(functions,
'reserveOrderNumbers');
6   const result = await reserve({ count });
7   return result.data.numbers;
8 }
```

ที่มา src/components/CartCheckoutModal.tsx บรรทัด 195 ถึง 215 ณ 17 กันยายน 2569

```
195
196     const response = await checkout({
197       idempotency_key: idempotencyKeyRef.current,
198       group_id: bundleId,
199       customer_name: customerName.trim(),
200       customer_phone: customerPhone.trim(),
201       customer_avatar: user.picture_url || '',
202       delivery_address: deliveryAddress.trim(),
203       location: location,
204       payment_method: paymentMethod,
205       cash_change_note: paymentMethod === 'cash' ? (cashChangeNote || '') : null,
206       orders: groupedItems.map((group) => ({
207         shop_id: group.shop.id,
208         items: group.items.map((it) => ({
209           menu_id: it.menu_id,
210           quantity: it.quantity,
211           selected_options: Array.isArray(it.selected_options) ? it.selected_options : [],
212           note: it.note || '',
213         })),
214       })),
215     });
```

getNextOrderNumbers เป็น helper จองเลขในโค้ดเดิม ส่วน checkout

ฉบับอัปเดตส่งคำขอไป checkoutOrder โดยมี idempotency_key และรายการแต่ละร้าน backend เป็นผู้จัดเลขและบันทึกออเดอร์ภายใน transaction แล้วส่งผลกลับหน้าจอ

คำขอใหม่ส่งรหัสเมนู จำนวน ตัวเลือก และข้อมูลส่งของ โดยไม่ส่ง food_subtotal หรือ total_amount เป็นยอดสุดท้ายที่ backend ต้องเชื่อถือ backend

อ่านราคาเมนูจากฐานข้อมูลเพื่อคำนวณเอง

แล้วบันทึกความสัมพันธ์ระหว่างคำขอกับออเดอร์ที่สร้าง

ข้อจำกัดที่ควรตรวจเพิ่มเติมใน functions/checkout.js ณ

เวลาตรวจคือเมื่อไม่พบตัวเลือกในเมนูยังมี fallback ไปใช้ opt.price จากคำขอ และผล transaction ที่ retry ต้องคืนออเดอร์เดิมอย่างถูกต้องในทุกกรณี การตรวจนี้เป็นข้อสังเกตจากโค้ด ไม่ใช่ผลทดลองโจมตีระบบจริง

ที่มา firestore.rules บรรทัด 4 ถึง 11 ณ 17 กันยายน 2569

```
4   function signedIn() { return request.auth != null; }
5   function access() { return
get(/databases/(database)/documents/access/(request.auth.uid)).data; }
6   function hasAccess() { return signedIn() &&
exists(/databases/(database)/documents/access/(request.auth.uid)); }
7   function admin() { return hasAccess() && access().role == 'admin'; }
8   function merchant(shop) { return hasAccess() && access().role == 'merchant' && access().shop_id
== shop; }
9   function participant(order) { return signedIn() && (order.customer_uid == request.auth.uid ||
merchant(order.shop_id) || admin()); }
10  function changedOnly(fields) { return
request.resource.data.diff(resource.data).affectedKeys().hasOnly(fields); }
11  function role() { return hasAccess() ? access().role : 'customer'; }
```

signedIn ตรวจสอบว่าคำขอมีตัวตนจาก Firebase หรือไม่ hasAccess ตรวจสอบว่าเอกสารสิทธิ์ของ uid มีอยู่จริง admin และ merchant ใช้เอกสารนั้นกำหนดอำนาจ โดย merchant ต้องเป็นร้านเดียวกับ shop ที่ร้องขอ

participant รวมสิทธิ์สามกรณี ได้แก่ ลูกค้าเจ้าของออเดอร์ ร้านผู้รับออเดอร์ และผู้ดูแลระบบ จึงเป็นจุดที่อธิบายได้ว่าระบบไม่ควรเปิดออเดอร์หรือแชทให้ทุกคนอ่าน changedOnly ใช้จำกัดว่าการอัปเดตเปลี่ยนแปลงได้เฉพาะช่องที่กำหนด

ตัวอย่างเชิงเหตุการณ์ ลูกค้า ก มี customer_uid ตรงกับบัญชี จึงอ่านออเดอร์ตนเองได้ ลูกค้า ข ไม่ตรงและไม่มีสิทธิ์ร้านหรือแอดมินจึงไม่ผ่านเงื่อนไข ร้าน ค ต้องมี access.shop_id ตรงกับ order.shop_id มิฉะนั้นยังถือว่าเป็นบุคคลภายนอกสำหรับออเดอร์นี้

การตรวจสอบสิทธิ์ใน Rules เป็นส่วนหนึ่งของระบบ ส่วน Cloud Functions ใช้ Admin SDK ต้องตรวจสอบสิทธิ์ใน handler เองด้วย จึงต้องทดสอบทั้งสองเส้นทาง และทำให้แหล่งกำหนดบทบาทสอดคล้องกัน

ภาคผนวก ข โค้ด **transaction** เมื่อจบออเดอร์

ที่มา functions/index.js บรรทัด 178 ถึง 195 ณ 17 กันยายน 2569

```
178     await db.runTransaction(async tx => {
179         const oSnap = await tx.get(orderRef);
180         if (!oSnap.exists) throw new functions.https.HttpsError('not-found', 'Order not found');
181         const current = oSnap.data();
182         if (current.status === 'completed') return;
183         if (current.status !== 'delivering' && current.status !== 'cooking') {
184             throw new functions.https.HttpsError('failed-precondition', 'Order must be cooking or
delivering');
185         }
186
187         // Firestore requires ALL reads before the first write.
188         const shopRef = db.collection('shops').doc(current.shop_id);
189         const shopSnap = await tx.get(shopRef);
190         if (!shopSnap.exists) throw new functions.https.HttpsError('not-found', 'Shop not found');
191         const settingsSnap = await tx.get(db.collection('system_settings').doc('general'));
192         const marker = db.collection('_completed_orders').doc(orderId);
193         const markerSnap = await tx.get(marker);
194         const quantities = new Map();
195         for (const item of current.items || []) {
```

transaction อ่านข้อมูลออเดอร์ล่าสุดอีกครั้งก่อนเปลี่ยนสถานะ หาก completed
แล้วจะไม่ทำซ้ำ หากยังไม่อยู่ใน cooking หรือ delivering จะปฏิเสธ จากนั้นอ่านร้าน การตั้งค่า
และ marker ก่อนเริ่มเขียนข้อมูลส่วนอื่นตามโค้ดถัดไป

การอ่านทั้งหมดก่อนเขียนเป็นข้อกำหนดสำคัญของรูปแบบ transaction ที่ใช้ ตัว marker
_completed_orders ช่วยบันทึกว่าเคยนับยอดขายของออเดอร์นี้แล้ว ส่วนสถานะออเดอร์ภายใน
transaction ช่วยควบคุมการจบซ้ำเมื่อมีค่าขอพร้อมกัน

ผล unit tests รองรับกรณีใช้งานและลองซ้ำในฟังก์ชันนี้ แต่การอนุญาตให้ client อัปเดต
completed โดยตรงยังเป็นเส้นทางที่ต้องปิดให้สอดคล้อง การมี transaction
ถูกต้องจุดเดียวไม่ได้ทำให้ทุกเส้นทางในระบบถูกบังคับผ่าน transaction อัตโนมัติ

ภาคผนวก ข โค้ดสรุปยอดตามธุรกรรมจริง

ที่มา src/lib/salesSummary.ts บรรทัด 19 ถึง 32 ณ 17 กันยายน 2569

```
19 }
20
21 export function summarizeSales(orders: Order[], transactions: CreditTransaction[], start: string,
end: string) {
22   const inRange = (key: string) => !!key && (!start || key >= start) && (!end || key <= end);
23   const completed = orders.filter(o => o.status === 'completed' && inRange(dateKey(o.completed_at
?? o.created_at)));
24   const ledger = transactions.filter(tx => inRange(dateKey(tx.created_at)));
25   const charges = chargedByOrder(transactions);
26   const sum = (values: number[]) => Math.round(values.reduce((a, b) => a + b, 0) * 100) / 100;
27   const food = sum(completed.map(o => o.food_subtotal ?? 0));
28   const delivery = sum(completed.map(o => o.delivery_fee ?? 0));
29   const collected = sum(completed.map(o => o.total_amount ?? ((o.food_subtotal ?? 0) +
(o.delivery_fee ?? 0))));
30   const gp = sum(completed.map(o => charges.get(o.id) ?? 0));
31   return {
32     completed, ledger, charges, food, delivery, collected, gp, net: sum([collected, -gp]),
```

completed เลือกออเดอร์ที่จบแล้วและอยู่ในช่วงวัน ใช้ completed_at
ก่อนเพื่อให้สอดคล้องกับวันทำงานสำเร็จ charges มาจากรายการหักจริงที่ผูก order_id
แล้วนำไปหาค่า gp ของแต่ละออเดอร์ ส่วน collected
ใช้ยอดรวมของออเดอร์หรือคำนวณจากอาหารบวกค่าส่งเมื่อไม่มีค่า

ตัวอย่างสมมติ ออเดอร์รับจากลูกค้า 130 บาท มี ledger หัก GP 6 บาท จะได้ net 124 บาท
หากไม่มีธุรกรรมหักใน map จะใช้ GP ศูนย์ การตีความรายงานจึงต้องดูทั้งสถานะออเดอร์และ
ledger ไม่คูณ 5% เองโดยไม่ตรวจการตั้งค่าและการหักจริง

แนวทางทดสอบคือสร้างออเดอร์ตรงขอบวันไทย และสร้างธุรกรรมที่อัตราหักไม่เท่ากัน
เพื่อตรวจว่าโปรแกรมนับตามหลักฐานจริง รวมทั้งตรวจช่วงวันที่ไม่มีข้อมูลว่าคืนศูนย์
ไม่ใช่ค่าค้างจากการค้นหาครั้งก่อน

เหตุใดเลือกหัวข้อนี้ ผู้จัดทำยังไม่พบหัวข้อเดิมที่ตรงความสนใจ
จึงเสนอปัญหาที่พบใกล้ตัวจากร้านในชุมชนโพสต์ขายอาหาร
และต้องการช่วยจัดข้อมูลให้ค้นหาและสั่งซื้อได้สะดวก
โดยควรยืนยันกับอาจารย์ว่าขอบเขตนี้ตรงข้อกำหนดรายวิชา

แตกต่างจากการโพสต์ขายอย่างไร มีข้อมูลร้านและเมนูแบบเป็นโครงสร้าง มีตะกร้า ออเดอร์
สถานะ และแชทที่อ้างอิงรายการเดียวกัน ประเด็นนี้เป็นความต่างของรูปแบบงาน
ไม่ใช่ผลพิสูจน์ว่าช่องทางหนึ่งดีกว่าอีกช่องทางในทุกกรณี

ใครจัดส่งและรับเงิน ร้านค้าเป็นผู้ดำเนินการเอง ระบบแสดงวิธีชำระเงินสดหรือสแกนจ่ายร้าน
และช่วยส่งข้อมูลระหว่างคู่ซื้อขาย ผู้ดูแลดูแลระบบและช่วยประสานงาน

เหตุใดเลือก Firebase โค้ดใช้บริการบัญชี ฐานข้อมูลที่มี listener และ Functions ร่วมกัน
ซึ่งเหมาะกับการเชื่อมข้อมูลสถานะและแชทของโครงการนี้ ขณะเดียวกันยังต้องออกแบบ Rules
การตรวจราคา และ transaction ให้ถูกต้อง

ระบบปลอดภัยแล้วหรือยัง มีการแยกสิทธิ์และผลทดสอบย่อยที่ผ่าน
ในระหว่างจัดทำรายงานมีโค้ด backend checkout และการกันค่าขอซ้ำเพิ่มแล้ว
บันทึกโครงการระบุว่แก่และ deploy หลายส่วน แต่ยังคงต้องตรวจ acceptance
บนอุปกรณ์จริงและกรณีขอบเขตก่อนรับรองพร้อมใช้งานทั่วไป

วัดความสำเร็จอย่างไร เริ่มจากเกณฑ์งานที่ทำได้ถูกต้องและสิทธิ์ข้อมูล
จากนั้นประเมินความสำเร็จของงานและความเข้าใจของลูกค้ากับร้านจริง
ยังไม่มีผลสำรวจที่จะอ้างอิงคะแนนความพึงพอใจ

ภาคผนวก ค แบบบันทึกทดสอบและประเมิน

แบบต่อไปนี้เป็นเครื่องมือสำหรับเก็บข้อมูลในอนาคต ยังไม่มีผลการทดลองจริงกรอกไว้
ผู้จัดทำสามารถทำสำเนาต่อหนึ่งกรณีและแนบภาพหลักฐาน
โดยใช้บัญชีทดสอบหรือปกปิดข้อมูลส่วนบุคคลก่อนเผยแพร่ในรายงาน

ช่องบันทึก	ข้อมูลที่ต้องกรอก
รหัสกรณีและวัตถุประสงค์	[ระบุ]
เวอร์ชัน วันที่ และสภาพแวดล้อม	[ระบุ commit / อุปกรณ์ / เครือข่าย]
บัญชีและข้อมูลเริ่มต้น	[ใช้รหัสบัญชีทดสอบ ไม่ใส่รหัสผ่าน]
ขั้นตอน	[ระบุทีละขั้น]
ผลที่คาดหวัง	[ระบุผลที่ตรวจสอบได้]
ผลจริงและหลักฐาน	[ระบุภาพ รหัสรายการ หรือ log ที่ตัดข้อมูลลับแล้ว]
สถานะและผู้ทดสอบ	[ผ่าน / ไม่ผ่าน / ยังไม่ทดสอบ]
การแก้และทดสอบซ้ำ	[ระบุเวอร์ชันและผลใหม่]

หัวข้อประเมินผู้ใช้ที่เสนอ: ความง่ายในการค้นเมนู ความเข้าใจราคากับค่าส่ง
ความชัดเจนของสถานะ และความสะดวกในการติดต่อร้าน ใช้มาตรวัดที่อธิบายความหมาย เช่น
1 น้อยที่สุด ถึง 5 มากที่สุด พร้อมคำถามปลายเปิดว่าติดขัดขั้นใด
ห้ามนำช่องว่างเหล่านี้ไปคำนวณเป็นคะแนนจนกว่าจะเก็บข้อมูลจริง